

Analysis: Binary Search Game

Test Set 1

Let $f(k)$ be the number of ways in which the game ends with a score $\geq k$. Then, the number of ways in which the game ends with a score of exactly k is $f(k) - f(k+1)$. Therefore, the answer is

$$\begin{aligned}
 \sum_{k=1}^{\mathbf{M}} k(f(k) - f(k+1)) &= \sum_{k=1}^{\mathbf{M}} kf(k) - \sum_{k=1}^{\mathbf{M}} kf(k+1) \\
 &= \sum_{k=1}^{\mathbf{M}} kf(k) - \sum_{k=2}^{\mathbf{M}+1} (k-1)f(k) \\
 &= f(1) - \mathbf{M}f(\mathbf{M}+1) + \sum_{k=2}^{\mathbf{M}} (k - (k-1))f(k) \\
 &= \sum_{k=1}^{\mathbf{M}} f(k).
 \end{aligned}$$

Notice in the last step that $f(\mathbf{M}+1) = 0$ by definition.

Let us now focus on calculating $f(k)$. For each card, we only care about whether its value is $\geq k$ or not. Let S be a fixed subset of cards which have value $\geq k$ (we will go through all $2^{\mathbf{N}}$ such subsets). We can assign a score of 0 (for cards outside of S) or 1 (for cards in S) to each consecutive subsequence of cells on the board that can be presented to players, in increasing order of length. Notice that only subsequences of cells whose length is a power of 2 and who are "aligned" can be presented to a player. That is, sequences containing cells of the form $i2^j + 1, i2^j + 2, \dots, i2^j + 2^j$ for each j between 0 and $\mathbf{L}$, inclusive, and each i between 0 and $2^{\mathbf{L}-j} - 1$, inclusive. There are $2^{\mathbf{L}} + 2^{\mathbf{L}-1} + \dots + 2 + 1 = 2^{\mathbf{L}+1} - 1$ valid combinations to go through. If we know the score for sequences of shorter lengths, we can decide the score for a sequence by checking whose turn it is (it is Alice's turn if and only if j and $\mathbf{L}$ have the same parity) and picking either the maximum or the minimum score of its two halves. If the score assigned by this to the full sequence is 0, then there is no way to finish with a score $\geq k$ with combination S . Otherwise, we need to compute the number of ways in which this can happen. This is simply the number of original card values that yield S , which is $(\mathbf{M} - k + 1)^t (k - 1)^{\mathbf{N}-t}$ where t is the number of cards that are $\geq k$ in S .

Overall, this solution takes $O(\mathbf{M} \times 2^{\mathbf{N}} \times 2^{\mathbf{L}})$ time, which is fast enough to pass Test Set 1.

Test Set 2

Note that the expression $(\mathbf{M} - k + 1)^t (k - 1)^{\mathbf{N}-t}$ from the previous solution, if we consider k the only variable, expands into a polynomial of degree at most $\mathbf{N}$. Consequently $f(k)$ is also a polynomial in k of degree at most $\mathbf{N}$ (as it is the sum of such polynomials).

If $f(k)$ is a polynomial of degree at most $\mathbf{N}$, then $g(x) = \sum_{k=1}^x f(k)$ is a polynomial in x of degree at most $\mathbf{N} + 1$. This is a consequence of [Faulhaber's formula](#).

Thus, the polynomial $g(x)$ can be fully determined by evaluating it at $\mathbf{N} + 2$ different points, say $g(0), g(1), \dots, g(\mathbf{N} + 1)$. We can evaluate those values in the same way as Test Set 1 and then use [interpolation](#) to find $g(\mathbf{M})$.

This yields an algorithm that takes time $O(\mathbf{N} \times 2^{\mathbf{N}} \times 2^{\mathbf{L}})$ to evaluate $\mathbf{N} + 2$ points of $f(k)$, plus the time complexity of evaluating $g(\mathbf{M})$ using interpolation. The latter can be done by evaluating the Lagrange polynomial in a straightforward way in $O(\mathbf{N}^2)$, and there are faster methods too, but this does not impact the overall runtime.

To speed up the calculation of $g(0), g(1), \dots, g(\mathbf{N} + 1)$, we find a way to reduce the $2^{\mathbf{N}}$ factor. Note that if a card's index appears only once (or not at all) in the board, we can do without fixing its value in advance (as being $\geq k$ or not). We can fix the values that are repeated in the board and then run the greedy algorithm but instead of scores 0 and 1 we compute, for each subsequence, the number of ways that it can be made 1 by assigning the values of cards whose indices are not repeated in the board. The decisions can still be made greedily, but instead of doing all the combinatorics at the end, we do them at each decision point.

With this optimization, we only need to run the greedy algorithm 2^X times per each value of g that we need, where X is the number of cards whose indices appear more than once in the board. There can be at most $2^{\mathbf{L}-1}$ of those cards, so we have reduced the time complexity of the first step to $O(\mathbf{N} \times 2^{2^{\mathbf{L}-1}} \times 2^{\mathbf{L}})$. This is finally enough to pass Test Set 2.

Analysis: Build-A-Pair

Test Set 1

With the total number of digits up to 8 we can iterate through all possible pairs of numbers we can construct from these digits and choose the optimal one. One way to do this is to iterate through all possible permutations of the given digits and then split every permutation into two in all possible ways as if it were a string. For every such a split we can construct the numbers and calculate the answer. Be careful though, as numbers should not start with 0, every split where 0 is the first digit in a number is not valid.

If we denote the length of D as N , then this takes $O(N! \times N^2)$ time.

Test Set 2

First we can calculate the lengths of each resulting number. These should be as close as possible, which means in case N is even, each number must have exactly $N/2$ digits, and if N is odd, then the larger number must have $\lceil N/2 \rceil$ digits and the smaller number must have $\lfloor N/2 \rfloor$ digits.

Further we will refer to the larger number as A and to the smaller number as B .

Now, let's deal with the case when N is odd. In such a case, we can iterate through all non-zero digits and try them as the first digit of A and B . Now we would want to make A as small as possible (as it is already larger than B), and B as large as possible. Luckily, these two goals are complementary: we can use the $\lfloor N/2 \rfloor$ smallest remaining digits to construct the rest of A (just take them in non-decreasing order), and use the rest to construct B (by taking them in non-increasing order). Notice that choosing the leading digits greedily is also possible, but care must be taken to not choose zero. This way of simply trying everything only takes $O(b^2 \times N)$ time anyway, where b is the base (10), which is fast enough for the limits of the problem.

The case when N is even is trickier, as just choosing the first digits does not give us a unique way to construct the rest of A and B , because there is no guarantee that A will be larger. However, we can use similar technique as part of the solution.

Note that if we have a prefix of length i $A_1 A_2 \dots A_i$ of A and prefix $B_1 B_2 \dots B_i$ of B , we can guarantee that A will be larger than B no matter what digits we use further if and only if there is a $k \leq i$ such that $A_j = B_j$ for all $1 \leq j < k$ and $A_k > B_k$. This gives us the following algorithm: choose all possible ways to construct $A_1 A_2 \dots A_{k-1}$ and $B_1 B_2 \dots B_{k-1}$, then iterate through all possible A_k and B_k , then construct the rest of A and B so that the former is as small as possible and the latter is as large as possible, using the same technique as above. For each of these possibilities, update the answer with the difference between the constructed numbers. Note, as before, that we need to make sure that the numbers we construct do not start with 0.

The time complexity of such an algorithm is $O(2^{N/2})$ for selecting $A_1 A_2 \dots A_{k-1}$ and $B_1 B_2 \dots B_{k-1}$ (these are pairs of equal digits, so we cannot have more than $N/2$ of them, and the order among them is irrelevant, as long as we do not start with zero), $O(b^2)$ for selecting A_k and B_k , and $O(N)$ for constructing the numbers once we got the prefixes. Overall this gives us $O(2^{N/2} \times b^2 \times N)$, which is really fast in practice with a good implementation.

The algorithm can be optimized further, but it is not required for the given constraints. There are ways to use the fact that number of digits is limited to reduce $2^{N/2}$ to something with b (the base) instead of N in the exponent. There is also a way to greedily solve the full problem in linear time. Moreover, if the input and output are given in [run-length encoding](#), it can be solved in linear time in that encoding of the input, which is $O(\log N)$.

Analysis: Fence Design

This problem asks us to find a [triangulation](#) of the given set of points that uses two particular edges. Many of the arguments that follow are referenced in the linked article, so you can use it as an introduction.

The first step to solve this problem is to notice some properties of the finished job. Let P be the input set of poles and F be a maximum set of fences with endpoints in P that do not intersect other than at endpoints.

1. The fences that are the edges of the [convex hull](#) of P are in F .

Proof: By definition, the edges of the convex hull do not intersect with any other possible fence. Therefore, they can be added to any set of fences that do not contain them without generating any invalid intersections, so any maximum set contains them all.

2. The size of F is at most $3N - 3 - c$ where c is the size of the convex hull of P .

Proof: Consider the graph where poles are nodes and fences in F are the edges. The graph is planar, so the generalized form of [Euler's formula](#) applies. The formula states that $N + A = K + |F|$, where A is the number of internal areas (not counting the outside) and K the number of connected components of the graph.

Notice that each edge on the convex hull is adjacent to exactly one internal area, and other edges are adjacent to at most two internal areas. Therefore, the sum of the number of sides of all internal areas is at least $3A$ and that is counting each convex hull edge once and each other edge at most twice, so $3A \leq c + 2(|F| - c) = 2|F| - c$, which means $A \leq (2|F| - c)/3$.

Replacing that in Euler's formula we obtain $N + (2|F| - c)/3 \geq K + |F|$. It follows that $3N + 2|F| - c \geq 3K + 3|F|$, and then $3N - c - 3K \geq |F|$. Since $K \geq 1$, we obtain $|F| \leq 3N - 3 - c$.

3. The size of F is at least $3N - 3 - c$.

Proof: By induction. The base case is when all points in P are on the convex hull. For that case, consider a set of all edges in the convex hull of P plus any [triangulation](#) of P . This set has exactly $2N - 3$ edges, which is equal to $3N - 3 - c$ when $N = c$, proving that there exists a set of at least that size without any invalid intersections.

If there exists a point p not on the convex hull of P , consider an optimal set of edges for $P - p$, which by inductive hypotheses has size $3(N - 1) - 3 - c$. Because $|F| = 3(N - 1) - c - 3$, and $3(N - 1) - c - 3K \geq |F|$ from (2), K , the number of connected components, must be exactly one. Similarly, every internal area must be a triangle, with all edges in the convex hull being adjacent to one of them and all edges not in the convex hull being adjacent to two of them. By definition, p is not on the convex hull. By the fact that there are no collinear triples, p is not in an existing edge. Thus, p is strictly contained in one of these triangles. Therefore, we can add edges from p to each of the 3 vertices of the triangle that contains p to get a solution for our problem of size $3(N - 1) - 3 - c + 3 = 3N - 3 - c$.

From (2) and (3) above we know the exact number of fences we need to build (given the size of the convex hull of the set of poles). Moreover, we know that such an answer contains the convex hull of P and every internal area delimited by fences in the output is a triangle. We can use this to devise algorithms to generate optimal sets.

Test Set 1

There are many possible solutions for Test Set 1. For example, the procedure in the proof of point (3) above shows how to solve the problem with no pre-placed fences. There are ad-hoc ways to get around the problems with pre-placed fences, but they take a lot of work, and there is something simpler.

The proofs above show that any maximal set of fences is also maximum (notice that we only used maximality in our reasoning). Therefore, we can simply add fences to a set as long as they do not intersect with any previously added fence. This algorithm can accommodate pre-placed fences quite easily: simply start with them. There are $O(N^2)$ potential fences to consider, and for each one we need to check whether it intersects any of the fences we already have. Since the solution overall is of size $O(N)$ this means checking $O(N^3)$ intersections. Checking a pair of line segments to see if they intersect can be done in constant time, which means this algorithm takes $O(N^3)$ time overall.

Test Set 2

As in Test Set 1, there are lots of algorithms that solve this problem without considering the pre-placed fences, but only some of them are easy enough to adapt to them. For example, the procedure from the proof of (3) can be implemented efficiently: if the order in which we process points is randomized and we keep the current triangles in a tree-like structure to perform the search for a triangle efficiently, we can get an expected $O(N \log N)$ time complexity. This leads to an algorithm similar to the [incremental algorithm to compute a Delauney triangulation](#).

Another option is to modify the [Graham Scan](#) algorithm to efficiently find the convex hull to keep not only a convex hull of the visited points, but also all the triangles for the points inside it.

Unfortunately, while the algorithms above can work with a lot of ad-hoc code to accommodate the pre-placed fences, they become really cumbersome. Below we present some better alternatives.

Let x be the intersection of both lines that are the infinite extension of a pre-placed fence. Since the fences don't intersect, x can occur on one of them, but not on both. Let us call any pre-placed fence that does not contain x f_1 , and the other fence f_2 . By their definitions, all of f_2 is on the same side of the line that extends f_1 . We can recognize which fence can be f_1 by comparing whether the [orientation](#) of the two endpoints of a potential f_1 and each endpoint of f_2 is the same (that is, checking whether both endpoints of the candidate f_2 lie on the same side of the line that goes through f_1).

Sweep-line

We can solve the problem without pre-placed fences with a [sweep-line](#) that considers the points in order of x-coordinate and maintains a convex hull of all the already seen points, as in the [monotone chain convex hull algorithm](#). When considering a new point p , we simply connect it to all points from the already-seen set that do not cause intersections from p . Notice that those points are a continuous range of the right side of the convex hull of the already-seen set. Therefore, we can find those points efficiently with [ternary searches](#) on its right side.

To accommodate pre-placed fences, we first rotate the plane to make f_2 vertical (possibly scaling everything to use only integers) and then run the sweep line algorithm only on points that are on the same side of the line that goes through f_1 as f_2 (including both endpoints of f_2 and neither endpoint of f_1). Since f_2 is now vertical, it will be included by the algorithm in the set when its second endpoint is processed. After that, we rotate everything again to make f_1 vertical, and start the algorithm from the set and convex hull we already have (the endpoints of f_1 will be the first two points that are processed in this second pass). As before, f_1 will be added naturally by the algorithm.

Notice the accommodation of the pre-placed fences only requires linear time work, so the overall algorithm, just as the version without pre-placed fences, requires $O(N \log N)$ time overall.

Divide and conquer

This divide and conquer algorithm also has a correlate to computing the convex hull. The idea is simple: divide the set of points with a line that goes through 2 points, compute the result of each side (both of which include those 2 points), and then combine.

Let us call the two recursive results P and Q . The convex hulls of both are convex polygons with a shared side. Then, we keep two current poles p and q . Initially, both p and q are on the same endpoint of the shared side. Both move away from that shared side: p through consecutive vertices of P and q through consecutive vertices of Q . Initially we move them both together. After that, let p_0 and q_0 be the previous pole where p and q were, respectively, and p_1 and q_1 be the next value for each (that is, p_0p and pp_1 are adjacent sides of the convex hull of P and q_0q and qq_1 are adjacent sides of the convex hull of Q). Then,

- If p_1q does not intersect p_0p , we set $p = p_1$.
- If q_1p does not intersect q_0q , we set $q = q_1$.
- Otherwise, we stop.

Each time we move one, we add the fence connecting the current p and q to the result. When we are done, we do the same starting from the other endpoint of the shared side.

To divide evenly, we can pick any point x , sort the other points by angle, and pick the median as y , dividing by the line that goes through x and y . Alternatively, we can pick points randomly. On average, the split of points would be about even (as we did in the proposed solution for [Juggle Struggle: Part 1](#)).

The work done to combine takes linear time and the work done to split takes either $O(N \log N)$ time for the sorting version or linear time for the randomized version. Using the [Master theorem](#) we can then see that using the randomized version, the overall algorithm takes $O(N \log N)$ time, and using the sorting version, the overall running time is $O(N \log^2 N)$.

Analysis: Square Free

Test Set 1

In general, we will build all grids with the correct row and column sums, then check each one to see if there are squares in them. In Test Set 1, the size of the input is at most 6×6 . This means that there are 2^{36} different possible grids, which is too many to generate, so we must make use of the row sum constraints. We do this by building the grid one cell at a time. As you fill in each cell, ensure that corresponding row and column sums are still possible. For example, if the row sum should be 3, but there are already 3 /s in this row, you cannot put another /. Similarly, if the desired row sum is 3, but there are already $R - 3$ \s in the row, you cannot put another \. Once we are done filling out the grid, we check if there are any squares in the grid that we have made. If there are no squares, we are done! Otherwise, we move on to the next possible grid. If we finish searching all possible grids and we have not found any, then it is impossible.

Are we sure this is fast enough? Each row either needs 0, 1, 2, 3, 4, 5, or 6 /s in it. There are $\binom{6}{0} = 1$ choices with 0 /s, $\binom{6}{1} = 6$ choices with 1 /s, $\binom{6}{2} = 15$ choices with 2 /s, $\binom{6}{3} = 20$ choices with 3 /s, $\binom{6}{4} = 15$ choices with 4 /s, $\binom{6}{5} = 6$ choices with 5 /s, and $\binom{6}{6} = 1$ choices with 6 /s. Thus, each row has at most 20 different valid choices. So this algorithm will explore at most 20^6 different grid. In practice, we will get nowhere near this bound. In particular, there is at most one valid row for the bottom row (since we must satisfy the column sum constraints). This alone reduces the search space to at most 20^5 , but this, too, is an overestimate since there will be a lot of pruning throughout the search.

Checking for squares can be done in several ways. The easiest is to iterate over all possible places that the top-most row of the square can be (and which consecutive columns the /\ are in). Then, for each possible size of square (1, 2, 3), just check the corresponding grid entries on the four sides of the square.

Test Set 2

The bounds for Test Set 2 are much too large to exhaustively search all grids, so we will need an insight. We call a grid that satisfies the row and column constraints a *configuration*. First, let's discuss how to find *some* configuration (it may or may not be square free). To do this, we will set this up as a graph and run [maximum flow](#) on it. There are R vertices that represent the rows and C vertices that represent the columns. We will put an edge with capacity 1 between every pair of (row, column) vertices. We will then connect the i -th row vertex to a super-row vertex with capacity S_i and connect the i -th column vertex to a super-column vertex with capacity D_i .

We now run flow through the network using the super-row vertex as the source and the super-column vertex as the sink. If the network is saturated (that is, every edge leaving the source has flow = capacity), then we have a solution. If there is flow in the edge between row r and column c , then the corresponding entry in the grid is a /, otherwise it is a \. If the flow is not saturated, then it is impossible to make a grid with the appropriate row and column sums. We leave a formal proof of the bijection between configurations and valid flows on the described network as an exercise.

At this point, we have *some* configuration, but it may have squares in it. We will discuss three different ways to produce a grid that is square free.

Lexicographically Smallest Configuration

In this solution, we notice that the [lexicographically smallest](#) configuration (treating $\backslash$ as smaller than $/$ reading in [row-major order](#)) is square-free! At first this is not obvious. However, think about two rows ($r_i < r_j$) and two columns ($c_k < c_\ell$). If

$(r_i, c_k) = /, (r_i, c_\ell) = \backslash, (r_j, c_k) = \backslash, (r_j, c_\ell) = /$, then this grid is not the lexicographically smallest configuration. Why? Because we can swap all four of those without breaking the row or column constraints, while giving us a smaller configuration ($(r_i, c_k) = \backslash, (r_i, c_\ell) = /, (r_j, c_k) = /, (r_j, c_\ell) = \backslash$). This means that a lexicographically smallest configuration has no squares in the grid, because the top-most row of a square must contain $/\backslash$ in the same columns in which the bottom-most row of the square has $\backslash/$.

How do we find the lexicographically smallest configuration? There are two ways: (1) give the edge between row r and column c a cost of $2^{r-1+(c-1)\mathbf{R}}$ and run [minimum-cost maximum-flow](#). This will guaranteed find the lexicographically smallest, but the edge-costs will be huge (up to $2^{\mathbf{R}\mathbf{C}-1}$). (2) We will run maximum flow iteratively. Say we have run maximum flow. Go through the edges in row-major order of their corresponding cell. If there is no flow going through an edge, then it is a $\backslash$. Great! This is the smallest this can be, so remove this edge from the graph to lock that in. If there is flow going through the edge, then we "unpush" the flow from that edge (decrease the flow from the sink to the column vertex to the row vertex to the source by 1) and temporarily change that edge's capacity to 0. We then run flow again. If the flow is still saturated, then we know there exists a configuration where this entry in the grid is a $\backslash$, so we can permanently remove this edge from the graph. If it is not saturated, then we must put that edge back into the graph, and this entry is forced to be a $/$.

Complexity-wise, the first run of maximum flow takes $O((\mathbf{RC})^2)$ time. Then, for each flow we run, we must only push one unit of flow through, which only takes a single augmenting path, so $O(\mathbf{RC})$ time per edge. Thus, in total, this takes $O((\mathbf{RC})^2)$ time. Note that you can also fully re-run maximum flow for each edge instead of only pushing one unit of flow with a sufficiently optimized flow implementation.

Minimum-Cost Maximum-Flow

In the Lexicographically Smallest Configuration solution, we described how you can use minimum-cost maximum flow to solve this problem with exponential edge costs. Here, we will solve the problem using only polynomial sized edge costs. This solution makes use of the same idea as the previous one: we want to avoid $(r_i, c_k) = /, (r_i, c_\ell) = \backslash, (r_j, c_k) = \backslash, (r_j, c_\ell) = /$, but other than that, we do not need a lexicographically smallest configuration. If we set the edge cost between row r and column c to be $r \times c$, then we will not get this configuration, since swapping all of these symbols does not affect the row/column sums and strictly decreases the total cost. The total cost is reduced by $i \times k + j \times \ell$ and increased by $i \times \ell + j \times k$, which is a net decrease of $(i - j)(k - \ell) > 0$ since $i < j$ and $k < \ell$.

Flip-Flop!

Another solution is to simply find some configuration, then check it for squares. If there are no squares, then we are done! If there is a square, consider the top-most and bottom-most row in the square. We will swap the top $/\backslash$ with the bottom $\backslash/$. This does not affect the row/column sums. In doing this, we have broken the current square, but may have created another square. We continue breaking squares until we do not find any. This process must eventually finish since at each swap, we are always making our grid lexicographically smaller. You can never do more than $O((\mathbf{RC})^2)$ swaps of this form.

Common Mistake

Be careful! Just because the sum of the S_i s is equal to the sum of the D_i s does not mean that a configuration exists! One example is the following input, where those sums coincide, yet there are no grids that meet all the per-row and per-column requirements.

```
4 6
2 0 6 6
4 2 2 2 2 2
```